

Workwarrior Technical Standard

WW-STD-001

Document Number	WW-STD-001	Version	0.1.0
Status	Draft	Date	2026-04-29
Maintained by	Babb Works	Website	www.babb.tel
Contact	workwarrior@babb.tel	Repository	github.com/babbworks/ww-standard

Abstract. This document defines the Workwarrior Technical Standard — the canonical specification for the Workwarrior shell productivity system developed and maintained by Babb Works. It covers architecture, directory structure, environment model, service contracts, browser UI, heuristic engine, AI integration, GitHub sync engine, UDA system, shell coding standards, and install policy. Implementors and contributors should treat this document as authoritative.

Released under the MIT License. Free to use, modify, and distribute. Maintained at github.com/babbworks/ww-standard. Contact: workwarrior@babb.tel.

Table of Contents

Contents

1	1. Overview	5
2	2. Architecture	5
2.1	Data Flow Summary	5
3	3. Directory Structure	6
4	4. Environment Variables	9
5	5. Profile Model	9
5.1	Structure	9
5.2	Lifecycle	10
5.3	Multiple Named Resources	10
5.4	Profile Groups	10
5.5	Backup/Restore Policy	10
6	6. Shell Bootstrap	10
6.1	bin/ww-init.sh	10
6.2	Shell Functions Injected	11
6.3	RC File Management	11
7	7. CLI Dispatcher	11
7.1	bin/ww	11
7.2	Service Discovery	12
7.3	Allow-Listed Commands	12
7.4	Security Constraint	12
8	8. Core Libraries	12
8.1	lib/core-utils.sh	12
8.2	lib/profile-manager.sh	12
8.3	lib/shell-integration.sh	12
8.4	lib/logging.sh	13
8.5	lib/taskwarrior-api.sh	13
8.6	lib/config-utils.sh	13
8.7	lib/config-loader.sh	13
9	9. Service Architecture	13
9.1	Contract	13
9.2	Minimal Service Template	14
9.3	Profile-Level Service Override	14
9.4	Service Tiers	14
10	10. Service Registry	14
11	11. Browser UI	16
11.1	Overview	16

11.2	Server Implementation	16
11.3	REST API	16
11.4	Security Model	16
11.5	Panels	17
11.6	CMD Input Routing	17
12	12. Heuristic Engine	17
12.1	Overview	17
12.2	Pipeline	17
12.3	Rule Structure	18
12.4	Confidence Levels by Phrasing Variation	18
12.5	Command Domains	18
12.6	Date Expression Normalization	18
12.7	Compiler	18
12.8	Self-Improvement Loop	19
13	13. AI Integration	19
13.1	Configuration	19
13.2	Modes	19
13.3	Provider Registry	20
13.4	Runtime Control	20
14	14. GitHub Sync Engine	20
14.1	Two Engines	20
14.2	Bugwarrior	20
14.3	ww github-sync	21
14.4	Field Mapping	21
14.5	Sync Engine Files	21
14.6	Conflict Resolution	22
14.7	Orphan Detection	22
14.8	Annotation/Comment Sync	22
14.9	Oracle Token Pattern	22
15	15. Weapons System	23
15.1	Sword	23
15.2	Gun	23
15.3	Next	23
15.4	Schedule	23
15.5	Planned Weapons	23
15.6	Weapon Constraints	23
16	16. UDA System	24
16.1	Overview	24
16.2	UDA Types	24
16.3	Source Classification	24
16.4	Management	24
16.5	Sync Permissions	24
16.6	Urgency Tuning	25
16.7	Density Scoring	25
17	17. Extensions	25
17.1	TaskWarrior Extensions Registry	25

17.2 Hook Events	25
17.3 TimeWarrior Extensions	26
18 18. Questions System	26
18.1 Template Structure	26
19 19. Export System	26
20 20. Fragility Register	27
21 21. Shell Coding Standards	27
21.1 Variable Naming	28
21.2 Function Template	28
22 22. Testing	28
22.1 BATS Tests	28
22.2 Python Tests (pytest)	28
22.3 Test Policy	28
23 23. Install Policy	29
23.1 Core Toolchain	29
23.2 Platform Matrix	29
23.3 Install Steps	29
23.4 Optional Extensions	29
24 24. Dependency Table	30

1 1. Overview

Workwarrior is a profile-based shell productivity system that wraps five open-source tools — TaskWarrior, TimeWarrior, JRNL, Hledger, and Bugwarrior — into a single unified command surface. The system provides:

- **Profile isolation** — each profile is an independent directory containing all tool data for one work context
- **Unified CLI** — a single `ww` dispatcher routes all commands to service scripts
- **Natural language input** — 627 compiled heuristic regex rules translate plain English to tool commands before optionally falling back to a local or remote LLM
- **Browser UI** — a locally-served web interface (Python 3 stdlib only) with 15+ panels, real-time SSE updates, and a unified command input
- **GitHub sync** — two complementary engines for pulling issues from 20+ services and bidirectional sync between individual tasks and GitHub issues
- **Extensible service architecture** — new services are executable scripts in `services/<category>/`, discovered at runtime with no registration step

Entry point: `bin/ww` (main CLI dispatcher)

Shell bootstrap: `bin/ww-init.sh` (sourced by `.bashrc/.zshrc` at shell start)

Canonical install path: `~/ww`

pt

2 2. Architecture

User types: `p-work`

- `lib/shell-integration.sh` sets `TASKRC`, `TASKDATA`, `TIMEWARRIORDB`, etc.
- all tools now operate on the work profile's data

User types: `task add "Ship API" due:friday`

- TaskWarrior writes to `profiles/work/.task/`
- on-modify hook starts TimeWarrior tracking

User types: `ww browser`

- Python3 HTTP server starts on `localhost:7777`
- serves static UI + REST API + SSE events
- reads profile data via same env vars

User types in browser CMD: `"add task review code and start tracking"`

- HeuristicEngine matches against 627 rules
- splits compound command on `"and"`
- executes: `task add review code + timew start review code`

2.1 Data Flow Summary

Shell init

`ww-init.sh` sourced

- `shell-integration.sh` defines `p-<name>` aliases and shell functions
- `ww` alias registered

Profile activation (p-work)

- > use_task_profile("work") called
- > 5 env vars exported
- > all subsequent tool calls use profile data

ww <command> [args]

- > bin/ww validates profile active (or allow-listed command)
- > discovers service script in services/<command>/
- > profile-level service checked first (overrides global)
- > executes service with remaining args

ww browser

- > services/browser/server.py starts ThreadingHTTPServer on :7777
- > SSE channel open for real-time profile events
- > REST API handles panel data and CMD execution
- > CMD input -> HeuristicEngine -> 627 rules -> AI fallback -> ww subcommand execution

pt

3 3. Directory Structure

bin/

ww	Main CLI dispatcher (all ww commands route here)
ww-init.sh	Shell bootstrap (sourced at shell start)
profile	Profile activation helper
custom	Custom service launcher
export	Export launcher
x	Delete launcher

lib/

core-utils.sh	Core bash libraries (sourced, not executed directly)
profile-manager.sh	WW_BASE resolution, profile validation, last-profile state
shell-integration.sh	Profile lifecycle: create, delete, backup, import, restore
logging.sh	Shell function injection, alias management, rc file writes
github-api.sh	log_info/log_success/log_warning/log_error/log_step
github-sync-state.sh	GitHub REST API via gh CLI
sync-pull.sh	Sync state persistence (JSON per task)
sync-push.sh	GitHub -> TaskWarrior pull
sync-bidirectional.sh	TaskWarrior -> GitHub push
sync-detector.sh	Orchestrates pull+push with conflict window
field-mapper.sh	Change detection (task side + GitHub side)
conflict-resolver.sh	Field mapping between TaskWarrior and GitHub formats
annotation-sync.sh	Last-write-wins conflict resolution with configurable window
sync-permissions.sh	TaskWarrior annotations <-> GitHub comments sync
config-loader.sh	Per-UDA sync permission tokens
config-utils.sh	Profile config loading (jrnل.yaml, ledgers.yaml, bugwarrior)
taskwarrior-api.sh	YAML parsing utilities
bugwarrior-integration.sh	tw_get_task, tw_update_task, tw_get_field wrappers
dependency-installer.sh	Bugwarrior config and pull helpers
	Platform-aware tool installer with version cards

installer-utils.sh	Install path resolution, shell RC block management
export-utils.sh	Profile data export (JSON, CSV, markdown)
delete-utils.sh	Profile deletion with backup safety
profile-stats.sh	Profile statistics and reporting
shortcode-registry.sh	Shortcut/alias registry read/write
error-handler.sh	GitHub error classification
services/	Service scripts (executable, discovered at runtime)
profile/	
create-ww-profile.sh	Full profile creation (dirs, taskrc, hooks, aliases)
manage-profiles.sh	list/info/delete/backup/import/restore
profile-tool.sh	Profile info display
urgency.sh	ww profile urgency --- urgency coefficient management
subservices/	
profile-uda.sh	ww profile uda --- full UDA management surface
profile-density.sh	ww profile density --- TWDensity integration
uda-manager.sh	Interactive UDA CRUD
custom/	
configure-journals.sh	ww custom journals
configure-ledgers.sh	ww custom ledgers
configure-tasks.sh	ww custom tasks
configure-times.sh	ww custom times
configure-issues.sh	ww custom issues / ww issues custom
github-sync.sh	ww issues push/pull/sync/enable/disable/status
questions/	
q.sh	Main questions dispatcher
handlers/	Per-service question handlers
templates/	YAML question templates
extensions/	
extensions.sh	ww extensions taskwarrior list/search/info
taskwarrior.py	Extension data fetcher
find/	
find.sh	ww find dispatcher
find.py	Python search implementation
groups/	
groups.sh	ww group list/show/create/add/remove/delete
models/	
models.sh	ww model list/providers/add/set-default
export/	
export.sh	ww export (JSON, CSV, markdown)
browser/	
server.py	Python3 HTTP server --- 15+ panels, SSE, CMD, heuristic engine
static/app.js	Browser UI --- sidebar, task editor, all panels
static/index.html	HTML shell --- dark terminal aesthetic
static/style.css	Styles
cmd/	
cmd.log	JSONL log of all CMD submissions
ctrl/	
ctrl.sh	ww ctrl status/ai-mode/prompt-ww/prompt-ai
x-delete/	
x.sh	ww x --- profile/data deletion with safety backups

servers/	Server management
scripts/	Utility scripts
find/	Cross-profile search
weapons/	
gun/	taskgun passthrough --- bulk task series generator
sword/	Task splitting into sequential subtasks with dependencies
scripts/	
compile-heuristics.py	Heuristic compilation
scan-taskwarrior-extensions.py	Extension registry scanner
functions/	Shell helper functions (sourced at init)
journals/	Journal helpers
ledgers/	Ledger helpers
tasks/	TaskWarrior helpers and extensions
times/	TimeWarrior helpers
issues/	Issue tracking helpers
tools/	
list/list.py	List management tool (ww list)
config/	
ai.yaml	AI mode, access points, preferred provider
cmd-heuristics.yaml	Compiled NL->command rules (627 rules)
cmd-heuristics-corpus.yaml	Synthetic corpus for heuristic validation
models.yaml	LLM provider/model registry
ctrl.yaml	CTRL service settings
groups.yaml	Profile group definitions
shortcuts.yaml	Shortcut definitions
profiles/	User profiles (created at runtime, gitignored)
<name>/	
.taskrc	TaskWarrior config (UDAs, reports, urgency)
.task/	TaskWarrior database
.timewarrior/	TimeWarrior database
journals/	Journal files
ledgers/	Hledger ledger files
jrnl.yaml	Journal name -> file mapping
ledgers.yaml	Ledger name -> file mapping
.config/	Service configs (bugwarrior)
system/	Control plane (not shipped to users)
CLAUDE.md	Primary agent context document
TASKS.md	Canonical task board
fragility-register.md	File fragility classifications
gates/	Gate contract templates
roles/	Agent role definitions
plans/	Planning documents
specs/	Specification documents
audits/	Audit outputs

workflows/ Process workflows
pt

4 4. Environment Variables

Set automatically by `use_task_profile()` when a profile is activated via `p-<name>`:

Variable	Value	Used By
WARRIOR_PROFILE	Active profile name (e.g. work)	All ww scripts
WORKWARRIOR_BASE	Profile base directory (~/ww/profiles/work)	All ww scripts
TASKRC	Path to profile <code>.taskrc</code>	TaskWarrior
TASKDATA	Path to profile <code>.task/</code>	TaskWarrior
TIMEWARRIORDB	Path to profile <code>.timewarrior/</code>	TimeWarrior

Rule: Nothing in the system hardcodes paths. All path resolution goes through these variables. A lib function or service that constructs a path by concatenating `~/ww/profiles/<name>/...` is a bug — it must use `$WORKWARRIOR_BASE`.

pt

5 5. Profile Model

5.1 Structure

```
profiles/<name>/
  .taskrc           TaskWarrior config (UDAs, reports, urgency coefficients, hooks)
  .task/           TaskWarrior database (taskchampion.sqlite3 + supporting files)
  .timewarrior/     TimeWarrior database (data/ + .timewarrior config)
  journals/         JRNL journal files
  ledgers/          Hledger ledger files
  jrnل.yaml         Journal name -> file path mapping
  ledgers.yaml      Ledger name -> file path mapping
  .config/          Per-profile service configs
    bugwarrior/     Bugwarrior config (bugwarriorrc or bugwarrior.tomل)
    taskcheck/      Taskcheck config
  services/         Per-profile service overrides (shadow global services)
  ai.yaml           Per-profile AI mode override (optional)
```

5.2 Lifecycle

```
ww profile create <name>      # Creates directory structure, writes .taskrc, injects hooks, c
ww profile list                # Lists all profiles with last-active timestamps
ww profile info <name>        # Shows profile details, resource inventory, sync state
ww profile delete <name>      # Marks deleted, creates safety backup
ww profile backup <name>      # Archives to profiles/<name>-backup-<timestamp>.tar.gz
ww profile import <archive>    # Creates new profile from archive
ww profile restore <archive>   # Replaces existing profile from archive
ww remove <name>              # Full removal: scrubs aliases, removes config references, opti
```

5.3 Multiple Named Resources

A profile can have multiple named journals and ledgers:

```
ww journal add <name>         # Creates journals/<name>.txt and adds mapping to jrnl.yaml
ww journal list               # Lists all journals in active profile
ww journal remove <name>      # Removes journal file and mapping
ww journal rename <old> <new> # Renames journal file and updates mapping

ww ledger add <name>          # Creates ledgers/<name>.journal and adds mapping to ledgers.ya
ww ledger list                # Lists all ledgers in active profile
```

5.4 Profile Groups

```
ww group create <name>        # Create a named group
ww group add <group> <profile> # Add profile to group
ww group list                 # List all groups
ww group show <name>          # Show group membership
ww group remove <group> <profile> # Remove profile from group
```

Groups enable batch operations: `ww export --group work` exports all profiles in the `work` group.

5.5 Backup/Restore Policy

Profile backup creates a `.tar.gz` archive of the entire profile directory. The archive is self-contained — `ww profile import <archive>` reconstructs the profile including all data, config, and resource mappings.

Safety backups are created automatically before `profile delete` and `ww remove`. The backup path is displayed on screen before deletion proceeds.

pt

6 6. Shell Bootstrap

6.1 bin/ww-init.sh

Sourced by the user's `.bashrc` or `.zshrc` at shell start. Responsibilities:

1. Sources `lib/shell-integration.sh`
2. Calls `ensure_shell_functions()` to verify injection is up to date
3. Restores the last active profile if `WARRIOR_PROFILE` is unset

6.2 Shell Functions Injected

Function	Description
<code>task [args]</code>	TaskWarrior with active profile's TASKRC and TASKDATA
<code>timew [args]</code>	TimeWarrior with active profile's TIMEWARRIORDB
<code>j [name] "text"</code>	JRNL entry — optional journal name, falls back to default
<code>l [name] <cmd></code>	Hledger command — optional ledger name, falls back to default
<code>i [args]</code>	Bugwarrior pull shorthand
<code>q <template></code>	Questions template runner
<code>list [args]</code>	Quick task list (ww list passthrough)
<code>search <term></code>	Cross-profile search
<code>p-<name></code>	Activate named profile — created per profile

6.3 RC File Management

`shell-integration.sh` manages a `# WW ALIASES` section in all detected shell rc files (`.bashrc`, `.zshrc`). Profile aliases are written to this section. `remove_profile_aliases()` removes only the entries for the specified profile. The section is idempotent — re-running never duplicates entries.

pt

7 7. CLI Dispatcher

7.1 bin/ww

The main entry point for all `ww` commands. Approximately 709 lines. Responsibilities:

1. Sources all `lib/*.sh` files
2. Checks profile active state (or allow-listed command flag)
3. Parses first argument as the service category
4. Discovers service script: checks `$WORKWARRIOR_BASE/services/<category>/` first, then `services/<category>/`
5. Validates known subcommands at the routing level for security-sensitive services
6. Executes the service with remaining arguments

7.2 Service Discovery

The dispatcher scans for executables matching the service category. Profile-level services shadow global services of the same name. If no match is found, the dispatcher prints a list of available services and exits 1.

7.3 Allow-Listed Commands

Some commands run without an active profile: `profile create`, `profile list`, `deps install`, `deps check`, `help`, `version`. All others require `WARRIOR_PROFILE` to be set.

7.4 Security Constraint

For the browser CMD endpoint, `ALLOWED_SUBCOMMANDS` is a frozenset of valid service names. POST /cmd requests must have a first token matching this set. No `sh -c`, no `eval`. Unknown subcommands return HTTP 400.

pt

8 8. Core Libraries

8.1 lib/core-utils.sh

- `resolve_ww_base()` — resolves `WORKWARRIOR_BASE` from `WARRIOR_PROFILE` if not set
- `validate_profile_active()` — exits 1 with message if no profile is active
- `get_last_profile()` / `set_last_profile()` — persists last-used profile name to `~/.ww_last_profile`
- `profile_exists(name)` — returns 0 if profile directory exists

8.2 lib/profile-manager.sh

- `create_profile(name)` — full profile creation: `mkdir` structure, write `.taskrc` template, inject TaskWarrior hooks, create `jrnل.yaml` and `ledgers.yaml`, write bugwarrior config template, call `create_profile_aliases()`
- `delete_profile(name)` — creates safety backup, removes profile directory
- `backup_profile(name)` — archives to `profiles/<name>-backup-<timestamp>.tar.gz`
- `import_profile(archive)` — extracts archive to `profiles/`, creates aliases
- `restore_profile(archive)` — replaces existing profile directory from archive

8.3 lib/shell-integration.sh

See Section 6.

- `use_task_profile(name)` — core activation: exports 5 env vars, sets last profile
- `create_profile_aliases(name)` — writes `p-<name>` to all rc files
- `remove_profile_aliases(name)` — removes all aliases for profile from rc files
- `get_ww_rc_files()` — returns list of shell rc files to write to
- `add_alias_to_section(name, value, [file])` — idempotent alias injection
- `ensure_shell_functions()` — verifies `ww-init.sh` source line present in all rc files

Re-source guard: `[[ -n "${SHELL_INTEGRATION_LOADED:-}" ]] && return 0`. Never `readonly` — normal user re-sourcing must not error.

8.4 lib/logging.sh

All user-facing output goes through these functions. Never use raw `echo` in lib or services.

Function	Output	Exit
<code>log_info "msg"</code>	<code>[INFO] msg to stdout</code>	—
<code>log_success "msg"</code>	<code>[[OK]] msg to stdout</code>	—
<code>log_warning "msg"</code>	<code>[WARN] msg to stderr</code>	—
<code>log_error "msg"</code>	<code>[ERROR] msg to stderr</code>	—
<code>log_step "msg"</code>	<code>[>-] msg to stdout</code>	—

8.5 lib/taskwarrior-api.sh

Wrappers for TaskWarrior operations that avoid direct `task` calls in lib functions (keeping testability):

- `tw_get_task(uuid)` — returns task JSON
- `tw_update_task(uuid, mods)` — applies modifications
- `tw_get_field(uuid, field)` — returns single field value

8.6 lib/config-utils.sh

YAML parsing utilities for `jrnl.yaml`, `ledgers.yaml`, and `ai.yaml`:

- `yaml_get(file, key)` — reads a single scalar value
- `yaml_get_mapping(file)` — returns key:value pairs from a mapping
- `yaml_set(file, key, value)` — writes or updates a value

8.7 lib/config-loader.sh

Loads and validates GitHub sync configuration from bugwarrior config:

- `init_github_sync_config()` — main entry point, locates config, loads, validates
- `load_github_sync_config(config_path)` — exports `GITHUB_LOGIN`, `GITHUB_USERNAME`, `GITHUB_TOKEN`, `GITHUB_PROJECT_TEMPLATE`
- `validate_github_sync_config()` — checks required fields, returns 1 with specific error if missing
- Oracle token pattern: `@oracle:eval:gh auth token` — evaluated at load time via `gh auth token`

pt

9 9. Service Architecture

9.1 Contract

Every service script must:

1. Start with `#!/usr/bin/env bash` and `set -euo pipefail`
2. Respond to `--help` / `-h` with a one-line description and usage example
3. Use exit codes: 0 success, 1 user error, 2 system/internal error
4. Log via `lib/logging.sh` functions — never raw `echo` for user-facing messages

- 5. Not write to profile directories directly — call lib functions
- 6. Be discoverable: placed at `services/<category>/` with executable permission

9.2 Minimal Service Template

```
#!/usr/bin/env bash
set -euo pipefail

source "$WORKWARRIOR_BASE/lib/logging.sh"

USAGE="ww myservice --- one-line description
Usage: ww myservice <subcommand> [args]
"

case "${1:-}" in
  --help|-h)  echo "$USAGE"; exit 0 ;;
  list)       # implementation ;;
  info)
    [[ -z "${2:-}" ]] && { log_error "info requires a name"; exit 1; }
    # implementation ;;
  *)
    log_error "Unknown subcommand: '${1:-}'"
    echo "$USAGE"
    exit 1
    ;;
esac
```

9.3 Profile-Level Service Override

Services at `profiles/<name>/services/<category>/` shadow global services. The dispatcher checks the profile path first. This enables per-profile service customizations without modifying global services.

9.4 Service Tiers

Tier	Description	Use Case
1 — Simple	Single-file, direct tool calls	Single-tool wrappers, simple data reads
2 — Compound	Multiple subcommands, uses lib functions	Lifecycle services (create/list/delete/backup)
3 — Complex	State management, external APIs, background processes	GitHub sync, browser server

pt

10 10. Service Registry

Domain	Commands	Description
ww profile	create, list, info, delete, backup, import, restore, uda, urgency, density	Profile lifecycle and configuration
ww journal	add, list, remove, rename	Named journal management
ww ledger	add, list, remove, rename	Named ledger management
ww group	list, create, show, add, remove, delete	Profile groups for batch operations
ww model	list, providers, show, add-provider, remove-provider, add-model, set-default, env, check	LLM provider and model registry
ww ctrl	status, ai-on, ai-off, ai-status, ai-mode, prompt-ww, prompt-ai	AI mode, prompt settings, UI config
ww find	<term> with filters	Cross-profile search across tasks, time, journals, ledgers
ww issues	sync, push, pull, status, enable, disable, custom, uda	GitHub two-way sync + Bugwarrior pull
ww custom	journals, ledgers, tasks, times, issues	Interactive configuration wizards
ww extensions	taskwarrior list/search/info/refresh	Extension registry
ww export	JSON, CSV, markdown	Profile data export
ww questions	list, new, delete, <template>	Template-based capture workflows
ww browser	start, stop, status, export	Locally-served web UI
ww remove	<profile>, -keep, -all, -archive-all, -delete-all	Profile removal with scrubbing
ww shortcut	list, info, add, remove	Shortcut/alias reference
ww deps	install, check	Dependency management
ww compile-heuristics	-verbose, -digest	Recompile NL->command rules
ww gun	<args>	Bulk task series (wraps taskgun)
ww sword	<task> -p N [-interval Nd]	Task splitting with dependency chains
ww next	—	CFS-inspired next-task recommendation
ww schedule	—	Auto-scheduler (wraps taskcheck)
ww mcp	install, status	MCP server for AI agent access
ww tui	install	taskwarrior-tui installer
ww list	—	List management tool

pt

11 11. Browser UI

11.1 Overview

`ww browser` starts a locally-served web interface on `http://localhost:7777`. No npm, no build step, no cloud, no accounts. Python 3 stdlib only.

```
ww browser                # Start on port 7777, open browser
ww browser --port 9090    # Custom port
ww browser --no-open      # Start without opening browser
ww browser stop           # Stop server
ww browser status         # Show running state
```

11.2 Server Implementation

- `services/browser/server.py` — single Python file implementing `ThreadingHTTPServer`
- Static files served from `services/browser/static/` on each request (no caching — changes visible on refresh)
- `ThreadingHTTPServer` handles SSE connections (which hold sockets open) without blocking concurrent POST requests
- SSE endpoint: `GET /events` — streams profile change events as `text/event-stream`

11.3 REST API

Method	Path	Description
GET	<code>/health</code>	Liveness check with profile name and version
GET	<code>/data/tasks</code>	Pending tasks for active profile
GET	<code>/data/time</code>	Time intervals and totals
GET	<code>/data/journal</code>	Recent journal entries (active journal)
GET	<code>/data/ledger</code>	Account balances and recent transactions
GET	<code>/data/ctrl</code>	AI settings and resolved provider
GET	<code>/data/models</code>	LLM provider registry
GET	<code>/data/profile</code>	Active profile info and resource lists
GET	<code>/data/groups</code>	Profile group definitions
GET	<code>/data/sync</code>	GitHub sync state for active profile
GET	<code>/data/questions</code>	Available question templates
POST	<code>/cmd</code>	Execute a <code>ww</code> subcommand
POST	<code>/profile/switch</code>	Switch active profile
GET	<code>/events</code>	SSE stream

11.4 Security Model

`ALLOWED_SUBCOMMANDS` frozenset: every POST `/cmd` request is validated — the first token must appear in the frozenset. Unknown subcommands return HTTP 400. No `sh -c`, no `eval`.

This bounds the attack surface: XSS in the UI can only invoke known `ww` subcommands. Known `ww` subcommands cannot exec arbitrary shell commands.

11.5 Panels

Panel	Data	Actions
Tasks	Full task list with UDAs	Inline edit, start/stop/done, add, annotate
Time	Today's total, weekly breakdown, recent intervals	Start, stop
Journals	Entry list with expand/collapse	New entry, journal selector dropdown
Ledgers	Account balances, recent transactions, income statement	New transaction, ledger selector
CMD	Natural language + direct command input	Route indicator (fast)/(settings)
CTRL	AI mode toggle	off / local-only / local+remote
Models	LLM provider and model registry	Add provider, set default
Groups	Profile group management	Create, add member
Sync	GitHub sync dashboard	Sync, push, status
Questions	Template browser	Run template
Profile	Profile info and resource management	Switch profile, resource list
Weapons bar	Gun, Sword, Next, Schedule icons	Opens weapon panel

11.6 CMD Input Routing

Input received

- > Compound check: contains "and"/"then"/"also"/"plus"?
- > Yes: split into segments
- > Each segment:
 - > HeuristicEngine.match(segment, threshold=0.8)
 - > If match: execute
 - > If no match: AI fallback if configured
 - > If no AI: return error with suggestion
- > Collect results, return to UI

pt

12 12. Heuristic Engine

12.1 Overview

627 compiled regex rules across 19 command domains. No network. No latency. Loaded at server startup from `config/cmd-heuristics.yaml`.

12.2 Pipeline

1. **Compound split** — input containing “and”, “then”, “also”, “plus” is split into segments

2. **Rule evaluation** — each segment tested against all 627 rules, highest confidence wins
3. **Threshold check** — match must exceed 0.8 confidence to accept
4. **AI fallback** — if no rule matches, input goes to configured LLM (if enabled)
5. **Execution** — matched commands executed against active profile

12.3 Rule Structure

Each rule contains: - `pattern` — compiled regex - `action_template` — output command template with capture group references - `confidence` — score (0.0–1.0) - `domain` — one of 19 command domains

12.4 Confidence Levels by Phrasing Variation

Variation	Confidence	Example
Passthrough	1.0	task add review budget
Imperative	0.95	add a task to review the budget
Declarative	0.90	I need a task for reviewing the budget
Interrogative	0.90	can you create a task to review the budget
Shorthand	0.90	task: review budget due friday
Verbose	0.85	I would like to add a new task for reviewing the budget

12.5 Command Domains

task · time · journal · ledger · profile · group · model · ctrl · service · issues · find · schedule · gun · next · mcp · browser · extensions · custom · shortcut

12.6 Date Expression Normalization

Input	Output
"tomorrow"	due:tomorrow
"next week"	due:1w
"friday"	due:friday
"end of month"	due:eom
"in 3 days"	due:3d

12.7 Compiler

`scripts/compile-heuristics.py` generates `config/cmd-heuristics.yaml`:

```
ww compile-heuristics           # Standard run
ww compile-heuristics --verbose  # Every rule with test results
ww compile-heuristics --digest   # + CMD log analysis -> new rules from AI hits
```

Compiler reads: `bin/ww` case branches, `config/shortcuts.yaml`, `config/command-syntax.yaml`, and optionally `services/cmd/cmd.log`.

Steps: 1. Scan command sources for command definitions 2. Generate regex patterns for each phrasing variation 3. Validate against synthetic corpus (`config/cmd-heuristics-corpus.yaml`) 4. Resolve conflicts (higher-confidence rule wins) 5. Fill coverage gaps 6. Write output

12.8 Self-Improvement Loop

Every CMD submission is logged to `services/cmd/cmd.log` as JSONL: `{input, route, output, success, timestamp}`.

`--digest` flag reads this log, finds successful AI translations that had no heuristic match, generates new rules from those translations. Run `ww compile-heuristics --digest` regularly to expand heuristic coverage from real usage.

pt

13 13. AI Integration

13.1 Configuration

```
# config/ai.yaml
mode: off           # off | local-only | local+remote
preferred_provider: ollama
access_points:
  cmd_ai: true      # Enable AI in CMD service
```

Per-profile override: `profiles/<name>/ai.yaml` with same structure. Profile config takes precedence over global.

13.2 Modes

Mode	Behavior
off	Heuristics only. No AI calls. Unmatched inputs return error.
local-only	Heuristics first, then ollama if no match. No data leaves machine.
local+remote	Heuristics -> local LLM -> remote provider fallback chain.

13.3 Provider Registry

```
# config/models.yaml
providers:
  - name: ollama
    type: ollama
    base_url: http://localhost:11434
  - name: openai
    type: openai
    api_key_env: OPENAI_API_KEY
models:
  default: llama3.2
  fallback_chain: [llama3.2, gpt-4o-mini]
```

CLI management:

```
ww model add-provider ollama ollama http://localhost:11434
ww model add-provider openai openai
ww model set-default llama3.2
ww model list
ww model check          # Test connectivity to all configured providers
```

13.4 Runtime Control

```
ww ctrl ai-on          # Enable (uses mode from config)
ww ctrl ai-off         # Disable
ww ctrl ai-status      # Show resolved state (global + profile + effective)
ww ctrl ai-mode local-only # Set mode
```

Browser CTRL panel mirrors all CLI controls. Changes take effect immediately — no restart required.

pt

14 14. GitHub Sync Engine

14.1 Two Engines

Bugwarrior — one-way pull from 20+ services into TaskWarrior.

ww github-sync — two-way bidirectional sync between individual tasks and GitHub issues.

These are complementary. Use both simultaneously.

14.2 Bugwarrior

Pulls issues from: GitHub, GitLab, Jira, Trello, Bitbucket, Taiga, Pagure, and 13+ more.

```
ww issues custom      # Interactive configuration wizard (per-profile)
i pull                # Pull from all configured services
i status              # Show sync state
```

Configuration stored in `profiles/<name>/.config/bugwarrior/bugwarriorrc`.

Injected UDAs on created tasks: `githubissue`, `githuburl`, `githubrepo`, `githubauthor` — classified as `[github]` source in the UDA registry.

14.3 ww github-sync

Two-way sync between individual TaskWarrior tasks and GitHub issues.

```
ww issues enable <task_id> <issue_num> <org/repo>    # Link task to issue
ww issues disable <task_id>                          # Unlink
ww issues sync                                         # Two-way sync all linked tasks
ww issues push                                         # Push local changes to GitHub only
ww issues pull                                         # Pull GitHub changes only
ww issues status                                       # Show sync state for all linked tasks
```

14.4 Field Mapping

Bidirectional (both directions):

TaskWarrior	GitHub Issue
<code>description</code>	title (truncated to 256 chars on push)
<code>status</code>	state (pending/started -> OPEN, completed/deleted -> CLOSED)
<code>priority</code>	labels (H -> priority:high, M -> priority:medium, L -> priority:low)
<code>tags</code>	labels (system tags excluded)
<code>annotations</code>	comments (with <code>[tw]</code> prefix to prevent loop)

Pull-only (GitHub -> TaskWarrior):

GitHub Issue	TaskWarrior UDA
Issue number	<code>githubissue</code>
Issue URL	<code>githuburl</code>
Repository	<code>githubrepo</code>
Author	<code>githubauthor</code>
<code>createdAt</code>	entry (first sync only)
<code>closedAt</code>	end (if status -> completed)
<code>updatedAt</code>	modified

14.5 Sync Engine Files

All 10 files classified HIGH FRAGILITY. Changes require extended risk brief.

File	Role
<code>lib/github-api.sh</code>	GitHub REST API via gh CLI: get/update/label/comment
<code>lib/github-sync-state.sh</code>	Sync state persistence — JSON per task in .config/ww-sync/
<code>lib/sync-detector.sh</code>	Change detection — compares current state to last sync state
<code>lib/sync-pull.sh</code>	GitHub -> TaskWarrior pull with orphan detection
<code>lib/sync-push.sh</code>	TaskWarrior -> GitHub push with label management
<code>lib/sync-bidirectional.sh</code>	Orchestrates pull+push with conflict window management
<code>lib/field-mapper.sh</code>	Field mapping between TW and GitHub formats
<code>lib/conflict-resolver.sh</code>	Last-write-wins with configurable window
<code>lib/annotation-sync.sh</code>	TW annotations <-> GitHub comments
<code>services/custom/github-sync.sh</code>	Service entry point and CLI dispatch

14.6 Conflict Resolution

Last-write-wins with a configurable conflict window (default: 60 seconds).

If both sides changed within the conflict window, the sync reports the conflict rather than silently overwriting. The conflict is surfaced in `ww issues status` with both values and timestamps.

Outside the conflict window: the more recently modified side wins.

14.7 Orphan Detection

If a linked GitHub issue is deleted, `sync_pull_issue()` receives `[not-found]` from the API. Rather than hard-erroring, it logs a warning and skips the task. The user is prompted to run `ww issues disable <uuid>` to clean up the link.

14.8 Annotation/Comment Sync

TaskWarrior annotations are pushed as GitHub comments prefixed with `[tw]`. GitHub comments prefixed with `[tw]` are skipped on pull (preventing sync loops). Non-prefixed GitHub comments are pulled as TaskWarrior annotations prefixed with `[github]`.

14.9 Oracle Token Pattern

Bugwarrior config stores: `github.token = @oracle:eval:gh auth token`

The oracle directive is evaluated at load time via `gh auth token`. The actual token is never written to the config file. `lib/config-loader.sh` detects and evaluates this pattern.

pt

15 15. Weapons System

Weapons are service-level tools that manipulate profile data in special ways — creating, slicing, and packaging tasks. They appear as a weapons bar in the browser sidebar.

15.1 Sword

Native to ww. Splits a single task into N sequential subtasks with dependency chains.

```
ww sword <task_id> -p <N> # Split into N parts
ww sword <task_id> -p <N> --interval <Nd> # N-day intervals between parts
ww sword <task_id> -p <N> --prefix "Phase" # Custom prefix
```

Each subtask receives: - Description: “Part N of: {original description}” - Parent task’s project and tags - Due date: offset by $N \times \text{interval}$ from now - Dependency: on previous subtask in the chain

Parent task is archived after split. The chain is strictly sequential — a subtask cannot be completed until all its dependencies are complete.

15.2 Gun

Wraps [taskgun](#) (Rust). Bulk task series generator.

```
ww gun <args> # Arguments passed through to taskgun
```

Respects active profile (reads TASKRC/TASKDATA from env). Returns taskgun output with error handling.

15.3 Next

Wraps the [next](#) binary. CFS-inspired next-task recommendation.

```
ww next # Recommends single optimal next task
```

Factors: TaskWarrior urgency score, deadline proximity, context signals.

15.4 Schedule

Wraps [taskcheck](#). Auto-scheduler for time blocks.

```
ww schedule
```

15.5 Planned Weapons

Weapon	Status
Bat	Planned
Fire	Planned
Slingshot	Planned

15.6 Weapon Constraints

All weapons: - Read TASKRC/TASKDATA from environment (profile isolation respected)
- Work via `POST /cmd` in the browser UI - Never modify data outside active profile scope -

Respond to --help

pt

16 16. UDA System

16.1 Overview

TaskWarrior User Defined Attributes are first-class in Workwarrior. A profile can carry 100+ UDAs. The browser UI renders all UDAs in the task inline editor dynamically.

16.2 UDA Types

Type	TaskWarrior declaration	Use
string	uda.<name>.type=string	Text values
numeric	uda.<name>.type=numeric	Numbers, billable hours
date	uda.<name>.type=date	Dates
duration	uda.<name>.type=duration	Estimated time

16.3 Source Classification

Badge	Source
[github]	Injected by Bugwarrior from issue tracking services
[extension:<name>]	Added by TaskWarrior extensions or hooks
[custom]	Defined by user via <code>ww profile uda add</code>

Extension-classified UDAs are protected from accidental deletion.

Extension	UDAs	Badge
TWDensity	density, densitywindow	[extension:twdensity]
taskcheck	estimated, time_map	[extension:taskcheck]

16.4 Management

```
ww profile uda list           # All UDAs with source badges
ww profile uda add <name>    # Interactive UDA creation wizard
ww profile uda remove <name> # Remove (blocked for extension UDAs)
ww profile uda group <name>  # Apply UDA template group
ww profile uda perm <name> nosync # Set sync permission token
ww profile uda perm <name> private # Mark as private (no AI access)
ww profile uda perm <name> noai  # Exclude from AI context
```

16.5 Sync Permissions

Permission tokens stored in `profiles/<name>/config/ww-sync-permissions.yaml`:

Token	Effect
nosync	Excluded from github-sync push
private	Excluded from AI context
noai	Excluded from AI context
readonly	Pull-only, never pushed

16.6 Urgency Tuning

```
ww profile urgency    # Interactive urgency coefficient tuner
```

Tunable coefficients: `urgency.due.coefficient`, `urgency.priority.coefficient`, `urgency.age.coefficient`, `urgency.project.coefficient`, `urgency.tags.coefficient`, per-UDA urgency weights.

Results written to profile's `.taskrc`. Per-profile tuning means different contexts can have different priority models.

16.7 Density Scoring

```
ww profile density    # TWDensity integration --- due-date density scoring
```

TWDensity extension injects `density` and `densitywindow` UDAs. Higher density = more tasks due in the window = higher urgency.

pt

17 17. Extensions

17.1 TaskWarrior Extensions Registry

```
ww extensions taskwarrior list          # Browse community extensions
ww extensions taskwarrior search <term> # Search by name or tag
ww extensions taskwarrior info <name>   # Show details
ww extensions taskwarrior refresh        # Update registry from upstream
```

Registry populated by `scripts/scan-taskwarrior-extensions.py` — scans the TaskWarrior community extension list and indexes metadata.

17.2 Hook Events

Event	When
on-launch	Every task invocation
on-add	New task creation
on-modify	Task field change
on-exit	After task command exits

Workwarrior's own integration uses `on-modify` to auto-start TimeWarrior when `task start` is called.

17.3 TimeWarrior Extensions

Python scripts that run against TimeWarrior data. Custom reports, summaries, integrations. Installed to `~/.timewarrior/extensions/`.

pt

18. Questions System

Template-based structured capture workflows. A question template is a YAML file defining a series of prompts that write to tasks, journal, or ledger.

```
q <template>           # Run a template (interactive prompts)
ww questions list      # List available templates
ww questions new       # Create a new template interactively
ww questions delete    # Remove a template
```

Templates in `services/questions/templates/`. Per-profile templates in `profiles/<name>/templates/` shadow global templates.

18.1 Template Structure

```
name: standup
description: Daily standup notes
prompts:
  - field: yesterday
    label: "What did you do yesterday?"
    target: journal
    journal: standup
  - field: today
    label: "What will you do today?"
    target: task
    project: standup
  - field: blockers
    label: "Any blockers?"
    target: journal
    journal: standup
```

pt

19. Export System

```
ww export json          # Full profile export as JSON
ww export csv           # Tasks as CSV
ww export markdown      # Tasks as Markdown table
ww export --group <g>  # Export all profiles in group
```

Export scopes: tasks only, time only, journals only, ledgers only, or all resources.

Implemented in `lib/export-utils.sh`. Output to stdout by default; `--output <path>` writes to file.

pt

20 20. Fragility Register

Classification	Files	Constraints
HIGH FRAGILITY	<code>lib/github-api.sh</code> , <code>lib/github-sync-state.sh</code> , <code>lib/sync-pull.sh</code> , <code>lib/sync-push.sh</code> , <code>lib/sync-bidirectional.sh</code> , <code>lib/field-mapper.sh</code> , <code>lib/sync-detector.sh</code> , <code>lib/conflict-resolver.sh</code> , <code>lib/annotation-sync.sh</code> , <code>services/custom/github-sync.sh</code>	Extended risk brief required. Verifier sign-off before merge. Integration tests required.
SERIALIZED	<code>bin/ww</code> , <code>lib/shell-integration.sh</code>	One writer at a time. Changes affect all users immediately.
NEVER COMMIT	<code>profiles/*/task/taskoversioncontrol3</code> , <code>profiles/*/config/</code> , <code>profiles/*/list/</code> , <code>*.sqlite3</code>	User data. Gitignored. Never in version control.

pt

21 21. Shell Coding Standards

Every script in the project must follow these rules. Violations are Gate B failures.

```
#!/usr/bin/env bash
set -euo pipefail
```

- **Bash, not sh.** `#!/usr/bin/env bash` on every script. `#!/bin/sh` is not acceptable.
- `set -euo pipefail` is the second line. No exceptions.
 - `-e`: exit on error
 - `-u`: exit on undefined variable
 - `-o pipefail`: pipe failure propagates
- **Error propagation via return codes**, not exit traps or `exit 1` in lib functions.
- **Logging via** `lib/logging.sh` functions. Never `echo` for user-facing messages in lib or services.
- **Absolute paths always.** Use `$WORKWARRIOR_BASE/...`, never relative paths.
- **Quote all variable expansions:** `"$var"` not `$var`. `"${array[@]}"` not `${array[@]}`.
- **Functions in snake_case.** No camelCase. No kebab-case.
- **All local variables declared with** `local`.
- **No cd in lib functions.** Use full paths.

- **Exit codes:** 0 = success, 1 = user error, 2 = system/internal error.

21.1 Variable Naming

Scope	Convention
Global env vars	UPPER_SNAKE_CASE
Local function vars	lower_snake_case with local
Function names	lower_snake_case

21.2 Function Template

```
function_name() {
  local arg1="${1:?function_name requires arg1}"
  local arg2="${2:-default_value}"
  local result

  # implementation

  echo "$result"
  return 0
}
```

pt

22 22. Testing

22.1 BATS Tests

```
bats tests/ # All BATS test suites
bats tests/test-models-service.bats # Specific suite
bats tests/test-profile-manager.bats
bats tests/test-shell-integration.bats
bats tests/test-github-sync.bats
```

BATS (Bash Automated Testing System) provides shell-native test assertions. Each test file covers one service or lib component.

22.2 Python Tests (pytest)

```
python3 -m pytest services/browser/ # Browser server and heuristic engine
```

Tests cover: HTTP endpoint behavior, heuristic rule matching, compound command splitting, AI fallback routing, SSE event emission.

22.3 Test Policy

- Tests run locally. No CI currently active.
- Every implementation task (Gate C) requires test execution before merge.
- Sync engine changes require integration tests against a test GitHub repository.

- Shell function changes require BATS suite updates.

pt

23 23. Install Policy

23.1 Core Toolchain

`ww deps install` is the canonical install path.

Tool	Minimum Version	Package
TaskWarrior	3.0.0	task
TimeWarrior	1.4.0	timew
JRNL	4.0.0	jrn1 (via pipx)
Hledger	1.30	hledger
Bugwarrior	1.22.0	bugwarrior (via pipx)
GitHub CLI	2.0.0	gh
Python 3	3.9	python3
pipx	any	pipx

23.2 Platform Matrix

Platform	Manager	Auto-install
macOS	brew	Yes
Debian/Ubuntu	apt	No (emits command)
Fedora/RHEL	dnf	No (emits command)
Arch/Manjaro	pacman	No (emits command)
Other	—	Emits manual URLs

23.3 Install Steps

```
git clone <repo-url> ~/ww
cd ~/ww
./install.sh
source ~/.bashrc    # or source ~/.zshrc
ww deps check
ww profile create work
p-work
```

23.4 Optional Extensions

```
ww tui install    # taskwarrior-tui (full-screen TUI)
ww mcp install    # MCP server (AI agent access)
```

MCP install requires `uv` (Python package manager). Auto-installed on macOS via brew.

pt

24 24. Dependency Table

Dependency	Type	Role	Upstream
TaskWarrior	Core tool	Task management, UDAs, hooks	taskwarrior.org
TimeWarrior	Core tool	Time tracking	timewarrior.net
JRNL	Core tool	Journal entries	jrnل.sh
Hledger	Core tool	Double-entry accounting	hledger.org
Bugwarrior	Core tool	Issue pull from 20+ services	github.com/GothenburgBitFactory/bugwarrior
GitHub CLI (gh)	Required	GitHub sync authentication and API	cli.github.com
Python 3 (≥3.9)	Required	Browser UI server	python.org
pipx	Required	JRNL and Bugwarrior isolation	pypa.github.io/pipx
taskgun	Weapon	Bulk task series (Gun weapon)	github.com/hamzamohdzubair/taskgun
taskcheck	Weapon	Auto-scheduler (Schedule weapon)	—
taskwarrior-tui	Optional	Full-screen TUI	github.com/kdheepak/taskwarrior-tui
taskwarrior-mcp	Optional	MCP server for AI agents	github.com/hnsstrk/taskwarrior-mcp
ollama	Optional AI	Local LLM provider	ollama.ai
TWDensity	Optional extension	Due-date density scoring	—

pt

Workwarrior Technical Standard v0.1.0 — Babb — workwarrior@babb.tel

Workwarrior Technical Standard WW-STD-001 v0.1.0 | Maintained by Babb Works
www.babb.tel | workwarrior@babb.tel | github.com/babbworks/ww-standard
Released under the MIT License — free to use, modify, and distribute. This document is the authoritative specification for the Workwarrior system.